A PROJECT REPORT

AUTOMATED REGRESSION TESTING OF THE SAUCEDEMO WEB APPLICATION USING PLAYWRIGHT AND JAVASCRIPT

Submitted by

MANREET GILL

(24BCS10479)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

CHANDIGARH UNIVERSITY

November 2026

BONAFIDE CERTIFICATE

Certified that this project report "Automated Regression Testing of the SauceDemo Web Application using Playwright and JavaScript" is the bonafide work of "MANREET GILL (24BCS10479)" who carried out the project work under my supervision.

SIGNATURE
Er. Gagandeep Singh
HEAD OF THE DEPARTMENT
Academic designation
Department of Computer Science and Engineering
Chandigarh University, NH-95 Chandigarh-Ludhiana Highway, Gharuan, Mohali, Punjab - 140413, India

SIGNATURE
Er. Hartej Singh
SUPERVISOR
Academic designation
Department of Computer Science and Engineering

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I would like to thank my project supervisor for the guidance, encouragement and useful suggestions given during this project. The regular reviews helped me to keep the project simple and to concentrate on tests that I can run and explain.

I am thankful to the Head of the Department and to the faculty of the Department of Computer Science and Engineering, Chandigarh University, for the facilities and the support needed to complete this work.

I also thank the people who build and share free tools such as Playwright, Node.js and Git, and the makers of the SauceDemo website, which gives students a safe public website for practising testing.

Declaration of tools: AI-based assistants were used for guidance on structuring the code and the documents. All code was run on my own computer, the results and screenshots in this report are from my own execution, and I have reviewed and understood the work.

Finally, I thank my family and friends for their constant support.

Manreet Gill

TABLE OF CONTENTS

ACKNOWLEDGEMENT ..... **iii**
TABLE OF CONTENTS ..... **iv**
LIST OF FIGURES ..... **vi**
LIST OF TABLES ..... **vii**
ABSTRACT ..... **viii**
GRAPHICAL ABSTRACT ..... **ix**
ABBREVIATIONS ..... **x**
Chapter 1: Introduction **1**
1.1 Identification of Client and Need 1
1.2 Relevant Contemporary Issues 1
1.3 Problem Identification 1
1.4 Task Identification 1
1.5 Timeline 2
1.6 Organization of the Report 2
Chapter 2: Literature Survey **3**
2.1 Timeline of the Reported Problem 3
2.2 Analysis of the Literature Consulted 3
2.3 Solutions Used by Others 4
2.4 Summary Linking the Literature with the Project 4
2.5 Problem Definition 4
2.6 Goals and Objectives 5
Chapter 3: Design Flow / Process **6**
3.1 Concept Generation 6
3.2 Evaluation and Selection of Specifications and Features 6
3.3 Design Constraints 6
3.4 Analysis and Feature Finalization Subject to Constraints 7
3.5 Design Flow: Alternative Designs 7
3.6 Best Design Selection 8
3.7 Implementation Plan 8
Chapter 4: Results Analysis and Validation **11**
4.1 Implementation of the Design Using Modern Engineering Tools 11
4.2 Execution and Results 12
4.3 HTML Report 13
4.4 Analysis of the Results 14
4.5 Problems Faced and How They Were Solved 15
4.6 Project Management and Communication 16

Chapter 5: Conclusion and Future Work.....18
5.1 Conclusion.....18
5.2 Deviation from Expected Results.....18
5.3 Limitations.....18
5.4 Future Work.....18
REFERENCES.....20
Appendix-1: User Manual.....21
A1.1 What you need.....21
A1.2 Installation.....21
A1.3 Running the tests.....21
A1.4 Seeing the report.....22
A1.5 Reading the results.....22
A1.6 Common problems and solutions.....23
Appendix-2: Source Code and Explanation.....24
A2.1 How the Files Work Together.....24
A2.2 package.json.....25
A2.3 .gitignore.....27
A2.4 package-lock.json.....28
A2.5 playwright.config.js.....30
A2.6 utils/testData.js.....32
A2.7 pages/LoginPage.js.....34
A2.8 pages/InventoryPage.js.....36
A2.9 pages/CartPage.js.....39
A2.10 pages/CheckoutPage.js.....41
A2.11 tests/login.spec.js.....43
A2.12 tests/products.spec.js.....46
A2.13 tests/cart.spec.js.....49
A2.14 tests/checkout.spec.js.....52
A2.15 tests/logout.spec.js.....56
A2.16 README.md.....57
A2.17 Quick Reference Tables.....58
Appendix-3: Achievements.....60

LIST OF FIGURES

Figure 1: Parts of the testing project.....9
Figure 2: What happens in one test run.....10
Figure 3: Terminal after npm test: 17 passed (18.6 s).....13
Figure 4: HTML report: summary and first tests.....14
Figure 5: HTML report: remaining tests.....14
Figure 6: Time of each test in the final run (grey: first test of a runner).....15
Figure 7: Error shown when the Playwright browser was not downloaded.....16
Figure 8: Running npm test (user manual, step 5).....22
Figure 9: HTML report at localhost:9323 (user manual, step 8).....22

LIST OF TABLES

Table 1: Project timeline....2
Table 2: How web testing tools developed....3
Table 3: Sources used, by type....3
Table 4: Simple comparison of web testing tools....4
Table 5: Requirements....6
Table 6: Design constraints....6
Table 7: Points for each tool (out of 5)....7
Table 8: Points for each design (out of 5)....8
Table 9: Tools used....11
Table 10: Test cases and results of the final run....12
Table 11: Summary of the final run....13
Table 12: Results by group....14
Table 13: Problems and solutions....15
Table 14: History of changes in the repository....16
Table 15: Troubleshooting....23
Table 16: What happens when TC02 runs....24
Table 17: What happens when TC14 runs....25
Table 18: Line-by-line explanation of package.json....26
Table 19: Line-by-line explanation of .gitignore....27
Table 20: Line-by-line explanation of package-lock.json....29
Table 21: Line-by-line explanation of playwright.config.js....31
Table 22: Line-by-line explanation of utils/testData.js....33
Table 23: Line-by-line explanation of pages/LoginPage.js....35
Table 24: Line-by-line explanation of pages/InventoryPage.js....37
Table 25: Line-by-line explanation of pages/CartPage.js....40
Table 26: Line-by-line explanation of pages/CheckoutPage.js....41
Table 27: Line-by-line explanation of tests/login.spec.js....44
Table 28: Line-by-line explanation of tests/products.spec.js....47
Table 29: Line-by-line explanation of tests/cart.spec.js....50
Table 30: Line-by-line explanation of tests/checkout.spec.js....53
Table 31: Line-by-line explanation of tests/logout.spec.js....56
Table 32: All actions in the page files....58
Table 33: Which test uses which files....59

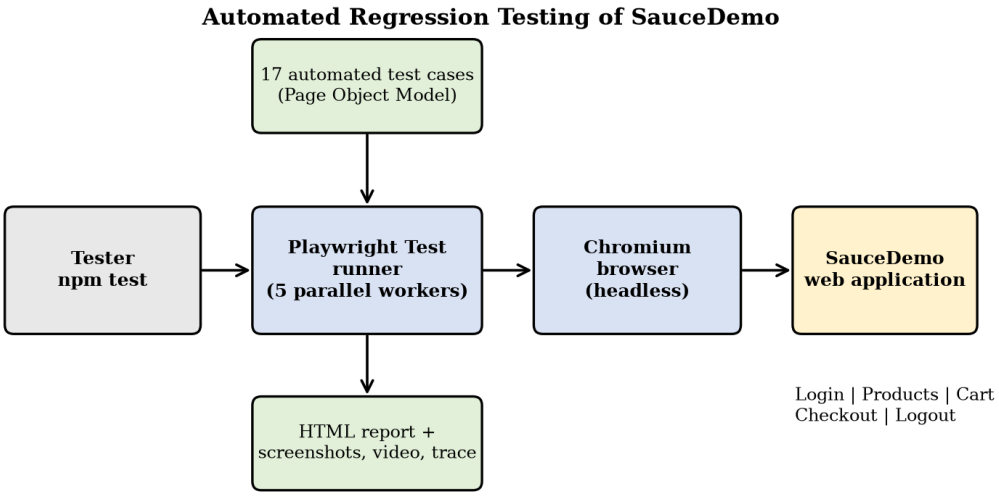
ABSTRACT

Checking a web shop by hand after every change is slow and boring, and people make mistakes. This project makes the computer do the checking. It tests SauceDemo, a public practice shop, using a free tool called Playwright and the JavaScript language. The project has 17 test cases. They check login, the product list and sorting, the cart, checkout and logout.

The code is organised in a simple way: one small file for each page of the shop, one file for the test data, and one file for each group of tests. Each test does one thing and then checks the result. The tests do not use fixed waiting times because the tool waits for the page by itself. After a run, the project creates a report that shows which tests passed or failed, and keeps pictures and videos for any test that fails.

In the final run all 17 tests passed and none failed. The run took 18.6 seconds. This report also explains every line of the code in plain words, so that the work can be understood and repeated by a beginner.

GRAPHICAL ABSTRACT



Graphical Abstract: Automated testing of SauceDemo with Playwright

ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
npm	Node Package Manager
QA	Quality Assurance
TC	Test Case
UI	User Interface
URL	Uniform Resource Locator (web address)
W3C	World Wide Web Consortium

CHAPTER 1
INTRODUCTION

1.1 Identification of Client and Need

The client in this project is a team that looks after a web shop. SauceDemo is used as an example of such a shop. It has the usual parts of every online shop: a login page, a list of products, a shopping cart, a form to pay and a logout button. If any of these parts stops working, customers cannot buy and the business loses money.

Every time the shop is changed, the team must check that the old parts still work. Doing this by hand means repeating the same clicks again and again. It takes a long time, it is tiring, and a person can easily forget a step. The need is a way to run the same checks quickly, every time, and to get a clear report of what passed and what failed.

1.2 Relevant Contemporary Issues

- Shops are updated often. Many teams now release small changes every few days, so there is very little time for manual checking (Humble and Farley, 2010).
- Web pages change quickly while they load. A test that waits for a fixed time can fail even when the page is correct, so tools that wait automatically are better.
- Mistakes found late cost more to fix than mistakes found early (Myers, Sandler and Badgett, 2011).
- Test code can become messy if the same details are repeated in every test. A tidy design keeps it easy to change (Fowler, 2013).
- Good testing tools are now free, so students and small teams can use them.

1.3 Problem Identification

Checking the login, products, cart, checkout and logout of a web shop by hand is slow and repetitive. The result depends on who does the checking, no proper record is kept, and it cannot be done quickly after every change. A student project also needs code that is simple enough to understand and explain.

1.4 Task Identification

The following tasks were identified:

- Study the SauceDemo website and decide which actions will be tested.
- Choose a testing tool and set up the project.
- Design a simple code structure: one file per page, one file for the data, one file per group of tests.

- Write the test cases with proper checks.
- Run the tests, create reports and study the results.
- Write the test plan, this report and a demo guide, and keep the work in a repository.

1.5 Timeline

The project was done in steps. The dates in Table 1 come from the run records and from the history of the repository. The first version had 23 tests. After a review it was made simpler, with 17 tests that are easier to explain and show.

Table 1: Project timeline

Step	Work done	Date
1	Study of the website, choice of tool, project set-up, first 23 tests	29 Sep 2026
2	First full run: 23 tests passed in 15.0 seconds; code put on GitHub	29 Sep 2026
3	Review and simplification: 17 tests kept (commit f4a58c3)	26 Oct 2026
4	Test plan written; browser download problem fixed	26 Oct 2026
5	Final run: 17 tests passed in 18.6 seconds; results added to the report	26 Oct 2026
6	Author name and package-lock.json added; work merged into the main branch	26 Oct 2026

1.6 Organization of the Report

The report has five chapters. Chapter 1 explains the need, the problem and the tasks. Chapter 2 looks at how testing tools developed and states the goals. Chapter 3 explains the design, the choices made and the plan. Chapter 4 shows how the project was built, the test results and what they mean. Chapter 5 gives the conclusion and future work. After the references there are three appendices: a step-by-step user manual, the complete code of the project with an explanation of every line (about two thirds of the report), and the achievements.

CHAPTER 2
LITERATURE SURVEY

2.1 Timeline of the Reported Problem

Testing software is an old activity. A well-known book on the subject first appeared in 1979 and explained that testing means looking for mistakes (Myers, Sandler and Badgett, 2011). When programs began to change all the time, checking again and again that old parts still work became expensive. This is called regression testing. Teams started to let the computer do it. Table 2 shows the main steps in the history of web testing tools.

Table 2: How web testing tools developed

Time	Event
1979	First edition of “The Art of Software Testing”.
2004	Selenium is created at ThoughtWorks. Tests can now control a real browser (Selenium Project, n.d.).
2010	The book “Continuous Delivery” links automatic tests with frequent, safe releases (Humble and Farley, 2010).
2013	The Page Object idea is described: keep the details of each page in one place (Fowler, 2013).
About 2017	Tools such as Puppeteer and Cypress become popular with JavaScript developers.
2018	W3C publishes the WebDriver standard for controlling browsers (W3C, 2018).
2020 onwards	Playwright is released by Microsoft. It waits for pages automatically and works with three browser engines (Microsoft, 2020).

2.2 Analysis of the Literature Consulted

Most of the useful material for this topic is tool documentation and books written by practitioners, not research papers, so a formal count of research papers was not done. Instead, the sources used in this project were grouped by type, as shown in Table 3.

Table 3: Sources used, by type

Type of source	Number	Examples
Official tool documentation	4	Playwright, Selenium, Cypress, Node.js
Books	2	Myers et al.; Humble and Farley
Standards and syllabi	2	W3C WebDriver; ISTQB
Online article	1	Fowler, PageObject
Website under test	1	SauceDemo
Total	10	

2.3 Solutions Used by Others

Several tools can test a website automatically. Selenium is the oldest and most widely known. It works with many languages, but it is only a controller: the user must add a test runner, reports and waiting rules. Puppeteer controls the Chromium browser from JavaScript, but it is also not a full testing tool. Cypress is a complete JavaScript testing tool that runs inside the browser. Playwright has everything in one package: it runs the tests, makes the checks, waits for pages, creates reports and works with three browser engines. Table 4 compares them in a simple way.

Table 4: Simple comparison of web testing tools

Feature	Selenium	Puppeteer	Cypress	Playwright
Language	Java, Python, JavaScript and others	JavaScript	JavaScript	JavaScript, Python, Java, .NET
Browsers	All main browsers	Mainly Chromium	Chromium, Firefox	Chromium, Firefox, WebKit
Waits for the page	User writes the waits	User writes the waits	Yes	Yes
Has its own test runner	No	No	Yes	Yes
Has reports	No	No	Basic	Yes (HTML report)
Cost	Free	Free	Free core	Free

The table is a simple summary of what each tool says in its own documentation. It is used only to help with the choice in Chapter 3.

2.4 Summary Linking the Literature with the Project

The reading gives three ideas that this project uses. First, repeated checks are best done by a computer. Second, test code stays tidy when the details of each page are kept in one place. Third, a modern tool that waits for pages and makes reports by itself means a beginner has to write less code. For these reasons the project uses Playwright with the Page Object idea on the public SauceDemo shop.

2.5 Problem Definition

To build a simple, tidy set of automatic tests for the main actions of the SauceDemo shop (login, products and sorting, cart, checkout and logout). The tests must run with one command, check the results properly, and produce reports that show clearly what passed and what failed.

2.6 Goals and Objectives

The goal is to replace repeated manual checking of the shop with automatic tests. The objectives are:

- To set up a clean, beginner-friendly Playwright and JavaScript project.
- To write 17 test cases for login, products and sorting, cart, checkout and logout.
- To keep the details of each page in one file, and the test data in another file.
- To check results without fixed waiting times.
- To create a terminal report and an HTML report, with pictures and videos for failures.
- To write a test plan, this report and a demo guide, and to keep the work in a repository.

CHAPTER 3
DESIGN FLOW / PROCESS

3.1 Concept Generation

Two decisions had to be made: which tool to use, and how to arrange the test code. Four tools from Chapter 2 were considered: Selenium, Puppeteer, Cypress and Playwright. For the code, three ideas were considered: separate scripts with everything repeated, page files with a special login helper, and page files with a simple “before each test” step.

3.2 Evaluation and Selection of Specifications and Features

The requirements come from the problem definition. Table 5 lists them. The first group says what the tests must check; the second group says what the project itself must be like.

Table 5: Requirements

No.	Type	Requirement
R1	Must check	Login with a correct user, a wrong password, a locked user and empty fields, and block the products page when not logged in.
R2	Must check	The shop lists 6 products, and sorting by name and by price puts the expected product first.
R3	Must check	Adding and removing items, the number on the cart icon, and the cart page.
R4	Must check	A complete purchase, and the messages for a missing first name and a missing postal code.
R5	Must check	Logout returns to the login page.
R6	Project quality	Simple code that a beginner can read, run and explain.
R7	Project quality	Tests that do not depend on each other and do not use fixed waiting times.
R8	Project quality	One command runs everything and takes well under a minute.
R9	Project quality	A readable report, with pictures and videos for failures.

3.3 Design Constraints

Table 6 lists the limits that were considered. Some do not matter much for a software testing project, and they are marked.

Table 6: Design constraints

Constraint	How it was considered
Regulations	Only SauceDemo is tested. It is a public site meant for practice. No other website is tested.
Economic	Only free tools are used, so the cost is zero. A normal laptop is enough.
Environmental	The tests run on one laptop for about twenty seconds, so very little energy is used.

Constraint	How it was considered
Health	No direct effect. Automatic testing reduces tiring repetitive work.
Manufacturability	Not applicable because the product is software.
Safety	The tests use only demo accounts. No real customer data, payment or live system is touched.
Professional	Clear names (TC01 to TC17), comments, version control and separate files for data, pages and tests.
Ethical	Results are reported exactly as seen. The tools used are declared. No real personal data is used.
Social	A shop that works well is better and safer for its customers.
Political	Not applicable.

3.4 Analysis and Feature Finalization Subject to Constraints

Keeping these limits in mind, the final features were chosen. Only the demo users of the public site are used. Only free tools are used. Tests that needed complicated code were avoided, so that every test can be explained and typed from memory. The first version had 23 tests. Tests that repeated another test (for example the last name check repeats the first name check) were removed, and 17 tests remained. Each important action is now covered once and clearly.

3.5 Design Flow: Alternative Designs

3.5.1 Alternative tools

The four tools were given points from 1 (poor) to 5 (very good) for the points in Table 7. Each point has an importance, shown as a percentage. Ease of learning and waiting for the page matter most for this project. The points are my own judgement, based on the tool documents and on requirements R6 to R9. They help in choosing; they are not measurements.

Table 7: Points for each tool (out of 5)

Point	Importance %	Selenium	Puppeteer	Cypress	Playwright
Easy to learn	20	3	3	4	4
Easy to set up	15	2	4	4	5
Has reports	15	2	2	4	5
Waits for the page	20	3	3	4	5
Many browsers	10	5	2	3	5
Free to use	10	5	5	5	5
Good help and documents	10	5	4	4	4
Total (out of 5)	100	3.30	3.20	4.00	4.70

3.5.2 Alternative designs of the test code

Three ways of arranging the test code were compared.

- Design 1: separate scripts. Every test contains everything, including the login steps. It is easy to read, but the same lines are repeated everywhere, so one change in the website needs many edits.
- Design 2: page files with a login helper. The details of each page are kept in one file and a helper logs in before each test. It avoids repetition, but the helper is an advanced idea and hard to explain. This was the first version, with 23 tests.
- Design 3: page files with a simple “before each test” step. The details of each page are kept in one file and each test group starts with a short step that logs in. It is tidy and easy to explain.

3.6 Best Design Selection

Table 8 gives points to the three designs. The importance favours easy reading and easy explaining, because the project has to be shown and explained, but it does not ignore easy changes later.

Table 8: Points for each design (out of 5)

Point	Importance %	Design 1	Design 2	Design 3
Easy to read for a beginner	25	4	3	5
Easy to change later	25	1	5	4
No repeated code	20	1	5	4
Easy to explain and show	30	4	2	5
Total (out of 5)	100	2.65	3.60	4.55

Playwright got the highest total (4.70) because it waits for pages, runs the tests and makes reports without extra tools. Design 3 got the highest total (4.55) because it is almost as tidy as Design 2 but much easier to explain. So Playwright with page files and a simple “before each test” step was chosen. Playwright scores at least as high as the other tools on every point, so the choice of tool does not depend on the importance values.

3.7 Implementation Plan

Figure 1 shows the parts of the project. The test files call the page files. The page files control a hidden browser through Playwright and visit SauceDemo. The test data and the settings are kept in their own files, and the reports collect the results.

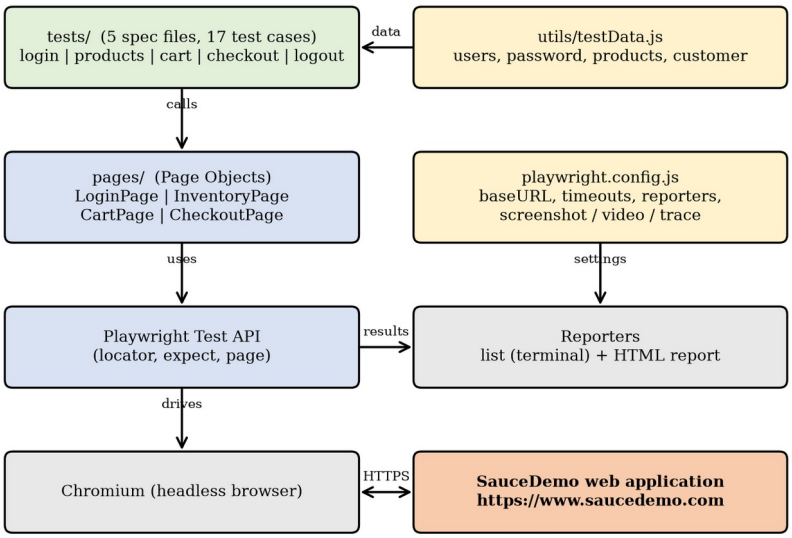


Figure 1: Parts of the testing project

Figure 2 shows what happens in one test run. In simple steps:

- Step 1: Playwright reads the settings and finds the test files.
- Step 2: It starts a browser and opens a fresh page for the test.
- Step 3: The “before each test” step opens the site and logs in.
- Step 4: The test does its actions: click, type, choose.
- Step 5: A check compares what the page shows with what is expected. It keeps trying for up to 5 seconds.
- Step 6: If the check is correct the test passes. If not, it fails and a picture, a video and a recording are saved.
- Step 7: All results go into the terminal report and the HTML report.

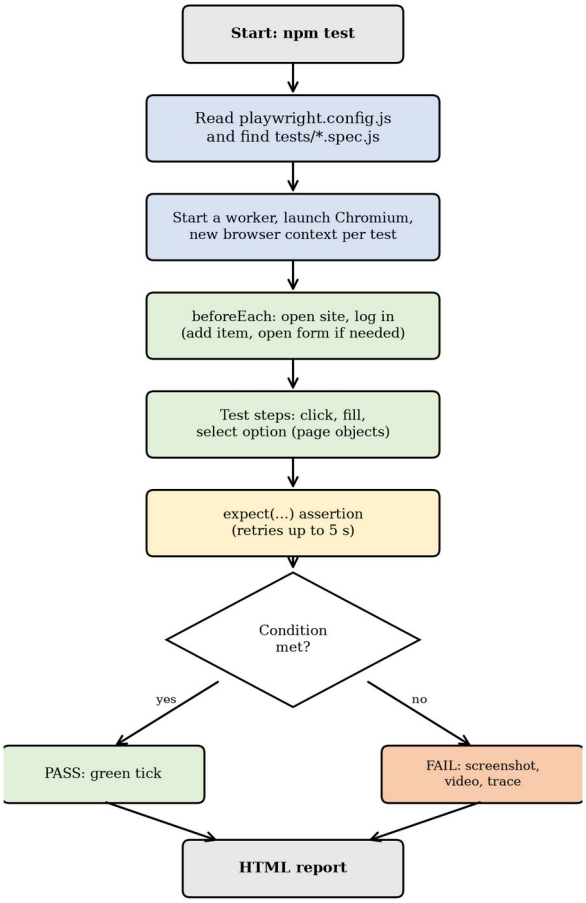


Figure 2: What happens in one test run

CHAPTER 4
RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of the Design Using Modern Engineering Tools

4.1.1 Tools used

The project was built and run on a Windows laptop with the tools in Table 9. Playwright downloads its own browser, so no extra browser or driver has to be installed by hand.

Table 9: Tools used

Item	Detail
Operating system	Windows
Runtime and package manager	Node.js and npm
Testing tool	@playwright/test (version 1.64.1 was installed; package.json asks for 1.63.0 or newer)
Browser	Chromium, started by Playwright without a window
Editor	Visual Studio Code with its built-in terminal
Version control	Git, with the repository on GitHub
Website under test	SauceDemo, https://www.saucedemo.com

4.1.2 Project folders

Each folder has one job. The list below shows the final project.

```
saucedemo-regression-tests/
|-- playwright.config.js   settings for all tests
|-- package.json           project details and shortcuts
|-- package-lock.json      exact versions of the tools
|-- pages/                 one file for each page of the shop
|-- tests/                 one file for each group of tests
|-- utils/testData.js      the data used by the tests
`-- docs/                  test plan and project report
```

4.1.3 How the code is arranged

The settings file tells Playwright where the tests are, how long to wait, which reports to make and what to keep when a test fails. Each page file holds what is known about one page of the shop: where the buttons and boxes are, and the actions a user can do. The test files do not search for buttons. They only call these actions and then check the result. If the shop changes a button, only one line in one page file has to be changed. All the code, with an explanation of every line, is in Appendix-2.

4.1.4 Test design

Every test follows the same pattern: do something, then check one thing. The checks wait up to 5 seconds, so no fixed waiting time is needed. Most test files start with a short

“before each test” step: the login tests open the login page, the products and cart tests log in, and the checkout tests also add the backpack and open the form. The logout file has one test, and that test logs in itself. Table 10 lists the 17 tests with their results.

Table 10: Test cases and results of the final run

No.	Group	Test	Expected and actual result	Status	Time (s)
TC01	Login	Valid login	Products page opens; title “Products”	Pass	8.8
TC02	Login	Wrong password	Error: user name and password do not match	Pass	2.6
TC03	Login	Locked-out user	Error: this user has been locked out	Pass	2.5
TC04	Login	Empty user name	Error: “Username is required”	Pass	1.1
TC05	Login	Empty password	Error: “Password is required”	Pass	1.1
TC06	Login	Products page without login	Error: only available after login	Pass	1.1
TC07	Products	Number of products	Exactly 6 products shown	Pass	8.5
TC08	Products	Sort name A to Z	First product: Sauce Labs Backpack	Pass	2.4
TC09	Products	Sort price low to high	First product: Sauce Labs Onesie	Pass	2.7
TC10	Cart	Add one item	Cart icon shows 1	Pass	8.9
TC11	Cart	Add two items	Cart icon shows 2	Pass	2.6
TC12	Cart	Remove item	Cart number disappears	Pass	2.7
TC13	Cart	Open the cart	Cart page lists the added item	Pass	1.2
TC14	Checkout	Complete purchase	Overview page opens; “Thank you for your order!”	Pass	8.8
TC15	Checkout	First name missing	Error: “First Name is required”	Pass	2.3
TC16	Checkout	Postal code missing	Error: “Postal Code is required”	Pass	2.5
TC17	Logout	Logout	Login button visible (login page shown)	Pass	9.1

4.2 Execution and Results

The tests were run with the command npm test. Playwright ran the 17 tests, several at the same time, using a hidden Chromium browser. Figure 3 shows the terminal at the end of the final run. Every test has a green tick and the last line says “17 passed (18.6s)”. Table 11 shows the summary.

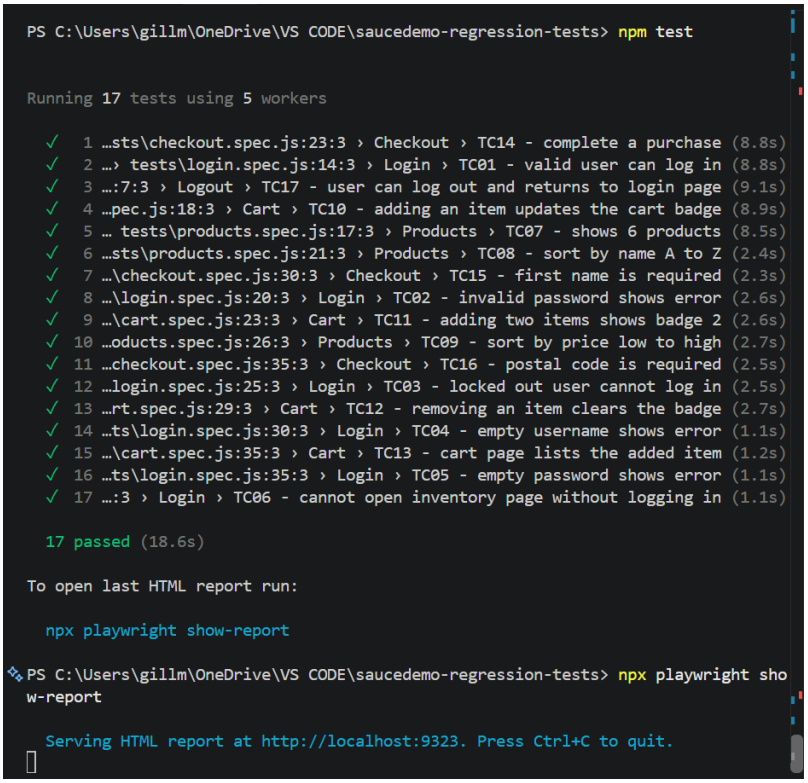


Figure 3: Terminal after npm test: 17 passed (18.6 s)

Table 11: Summary of the final run

Item	Result
Browser	Chromium, without a window
Tests run at the same time	5
Tests run	17
Passed	17
Failed / unstable / skipped	0 / 0 / 0
Total time	18.6 seconds
Date of the run	26 October 2026

4.3 HTML Report

After the run, the command npm run report shows the HTML report at http://localhost:9323. Figure 4 shows the top of the report: All 17, Passed 17, Failed 0, Flaky 0 and Skipped 0. Flaky means a test that passes sometimes and fails sometimes. The tests are listed by file with their times. Figure 5 shows the rest of the list.

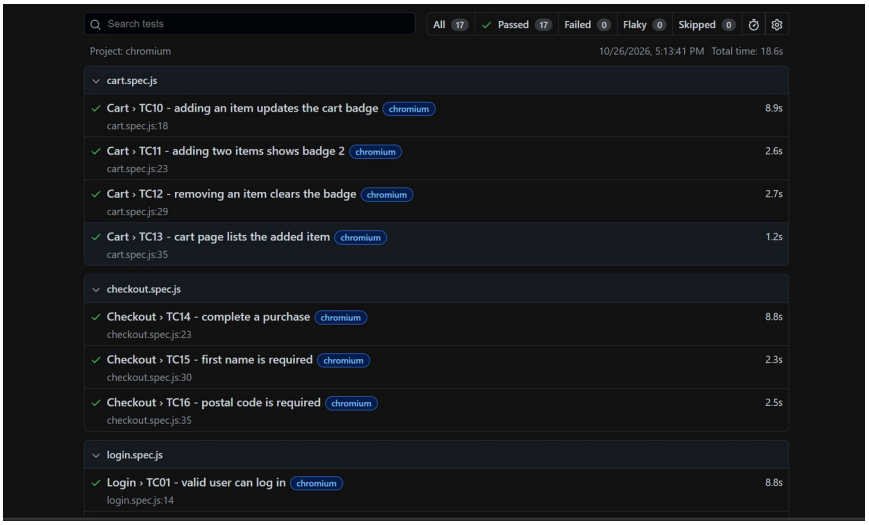


Figure 4: HTML report: summary and first tests

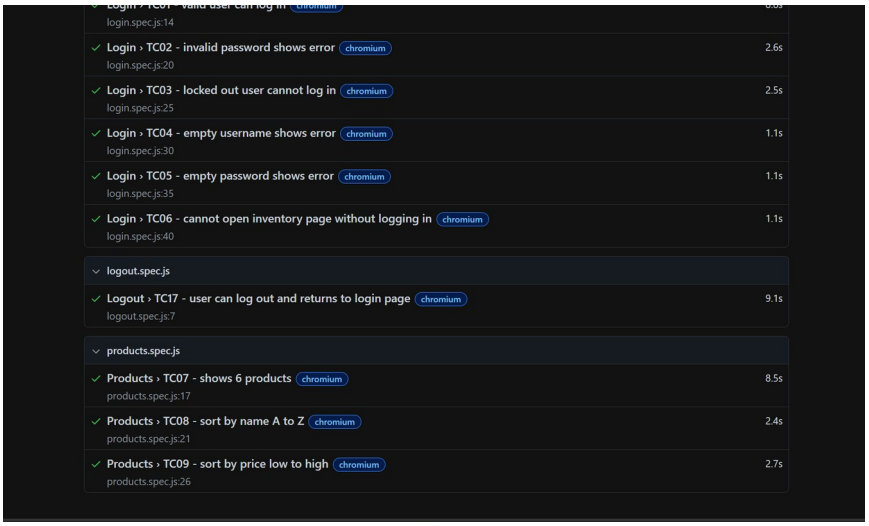


Figure 5: HTML report: remaining tests

4.4 Analysis of the Results

4.4.1 Results by group

Table 12 groups the results. Every test in every group passed. The sum of the times is bigger than 18.6 seconds because several tests ran at the same time.

Table 12: Results by group

Group	Tests	Passed	Failed	Sum of times (s)	Average (s)
Login	6	6	0	17.2	2.9
Products	3	3	0	13.6	4.5
Cart	4	4	0	15.4	3.8
Checkout	3	3	0	13.6	4.5
Logout	1	1	0	9.1	9.1

4.4.2 Time taken by each test

Figure 6 shows the time of each test. Five tests took 8.5 to 9.1 seconds (TC01, TC07, TC10, TC14 and TC17). The others took 1.1 to 2.7 seconds. The five slow ones were the first test started in each of the five runners. The reason is most likely that the browser has to start and load the first page, which happens only once for each runner. This is my explanation of the pattern, not a separate measurement.

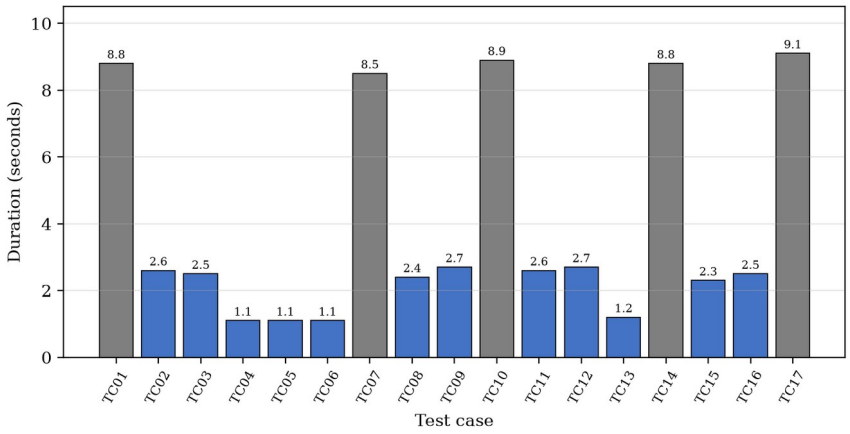


Figure 6: Time of each test in the final run (grey: first test of a runner)

4.4.3 Checking that the tests are meaningful

- Tests with correct input (TC01, TC14, TC17) show that the normal actions work: login opens the products page, a purchase ends with a thank-you message, and logout returns to the login page.
- Tests with wrong input (TC02 to TC06, TC15, TC16) show that mistakes are refused with the right message.
- Exact values are checked: 6 products, the first product after each sort, the address of the overview page and the thank-you message.
- Every test starts on a fresh page with an empty cart, so the order of the tests does not matter.

A good way to see that a test can really fail is to change one expected word, for example “Products” to “Product” in TC01, and run it again. It should fail and the report should show a picture. This is recommended before a demonstration.

4.5 Problems Faced and How They Were Solved

Table 13: Problems and solutions

Problem	Solution
npm said “Could not read package.json”	The zip file had opened into a folder inside a folder, so npm was in the wrong place. The terminal was moved into the inner folder that has package.json.

Problem	Solution
All tests failed at once with “Executable doesn’t exist” (Figure 7)	The browser needed by the installed Playwright version had not been downloaded. The command npx playwright install chromium fixed it. Those runs were not counted as results.
git push gave an error 400	The web address of the repository still had a placeholder name. It was corrected.
git push was rejected (“fetch first”)	The new repository was not empty. The project was sent again with a forced push, which was safe here.
git pull said “Unlink of file failed”	A Word file in the folder was open and the folder was being synchronised by OneDrive. Closing the file and pausing synchronisation solved it.
The project was hard to explain	The extra helper and the sorting mathematics were removed and 23 tests became 17 simple tests.

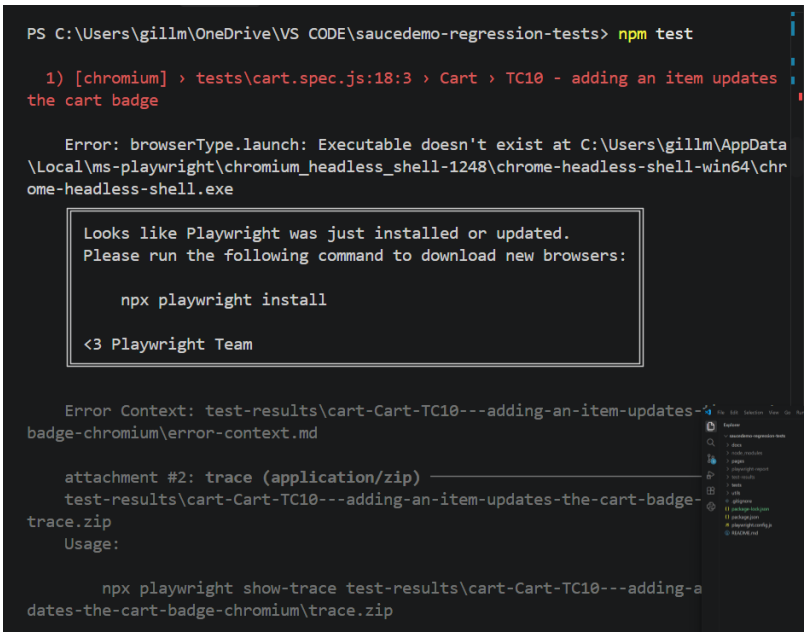


Figure 7: Error shown when the Playwright browser was not downloaded

4.6 Project Management and Communication

The work was managed with Git. Each change was saved with a message that says what changed, and the repository was put on GitHub so that it could be reviewed. Table 14 shows the history of the main branch. The review comments led to the simpler suite, the test plan and the small clean-up changes.

Table 14: History of changes in the repository

Change	Date	Message
fa51b4a	29 Sep 2026	SauceDemo regression tests (first version, 23 tests)
f4a58c3	26 Oct 2026	Simplify suite to 17 easy-to-explain tests; remove fixture and sorting maths
d4d1635	26 Oct 2026	Add test plan and test cases
cdda81f	26 Oct 2026	Fill test plan with real results (17 passed, 0 failed, 18.6 s)
deed233	26 Oct 2026	Add project report
b0e6a22	26 Oct 2026	Set author in package.json

Change	Date	Message
735fc33	26 Oct 2026	Add package-lock.json so installs use the same versions

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The project built a set of automatic tests for the SauceDemo web shop using Playwright and JavaScript. It has 17 test cases for login, products and sorting, cart, checkout and logout. In the final run all 17 tests passed, none failed, and the whole run took 18.6 seconds. This means the main parts of the shop can be checked again in less than twenty seconds after any change.

The objectives were met. The project is clean and simple, the tests wait for pages without fixed delays, the reports give clear proof, and the work is explained in a test plan, this report and a demo guide, and kept in a repository. One lesson was that a simple design that can be explained is better for a student project than a clever one. The first version with a helper was replaced by a simple “before each test” step without losing any main test.

5.2 Deviation from Expected Results

The test results were as expected: no test failed in the final run. Two things differed from the first plan. First, the first version had 23 tests and was reduced to 17 after the review. This gives up some repeated checks (last name check, two more sort orders, cart empty after an order) in return for simplicity. Second, early runs on the new copy of the project failed because the browser had not been downloaded. These were set-up problems, not test results, and one command solved them.

5.3 Limitations

- Only the Chromium browser is used, and only two demo users.
- The sort tests check only the first product, not the whole list.
- Some tests depend on the demo data of the shop (6 products and the first product after each sort). If the shop data changes, the tests must be updated.
- Internet access to the public SauceDemo site is needed.

5.4 Future Work

- Add the Firefox and WebKit browsers.
- Test the other demo users, such as problem_user and performance_glitch_user.
- Add back the removed tests in a simple form: sort Z to A and high to low, last name check, and cart empty after an order.

- Run the tests automatically whenever the code is changed, using GitHub Actions.
- Compare the whole sorted list, and add tests of the shop’s data services.

REFERENCES

1. Cypress.io (n.d.) Cypress Documentation. Available at: https://docs.cypress.io
2. Fowler, M. (2013) ‘PageObject’, martinfowler.com. Available at: https://martinfowler.com/bliki/PageObject.html
3. Humble, J. and Farley, D. (2010) Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley.
4. ISTQB (2023) Certified Tester Foundation Level Syllabus. International Software Testing Qualifications Board.
5. Microsoft (2020) Playwright Documentation. Available at: https://playwright.dev
6. Myers, G.J., Sandler, C. and Badgett, T. (2011) The Art of Software Testing. 3rd edn. Hoboken, NJ: John Wiley & Sons.
7. OpenJS Foundation (n.d.) Node.js Documentation. Available at: https://nodejs.org/docs
8. Sauce Labs (n.d.) SauceDemo (Swag Labs) demonstration application. Available at: https://www.saucedemo.com
9. Selenium Project (n.d.) Selenium Documentation. Available at: https://www.selenium.dev/documentation/
10. W3C (2018) WebDriver, W3C Recommendation. Available at: https://www.w3.org/TR/webdriver/

APPENDIX-1
USER MANUAL

This manual shows how to set up and run the project on a Windows computer. Type all commands in the terminal of Visual Studio Code (Terminal, then New Terminal), inside the project folder.

A1.1 What you need

- Node.js (the LTS version) from nodejs.org. It also installs npm.
- Git from git-scm.com, to download the code from GitHub.
- Visual Studio Code (recommended) and an internet connection.

A1.2 Installation

Step 1: Download the project: git clone https://github.com/gillmreet01/saucedemo-regression-tests.git

Step 2: Go into the folder: cd saucedemo-regression-tests

Step 3: Install the tools the project needs: npm install

Step 4: Download the browser used by the tests: npx playwright install chromium

If step 4 is skipped, the tests fail at once with “Executable doesn’t exist” (Figure 7). Running the command in step 4 fixes it.

A1.3 Running the tests

Step 5: Run all 17 tests: npm test. The terminal lists every test with a green tick and ends with “17 passed” (Figure 8).

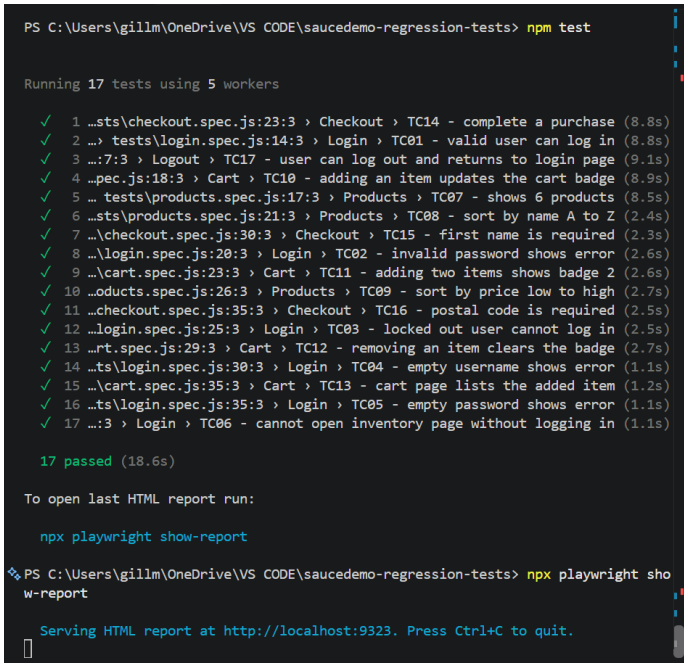


Figure 8: Running npm test (user manual, step 5)

Step 6: To watch the browser while the tests run: npm run test:headed

Step 7: To run one group only: npm run test:login, test:products, test:cart, test:checkout or test:logout.

A1.4 Seeing the report

Step 8: Open the HTML report: npm run report. The terminal says “Serving HTML report at http://localhost:9323”. Open this address in a browser (Figure 9).

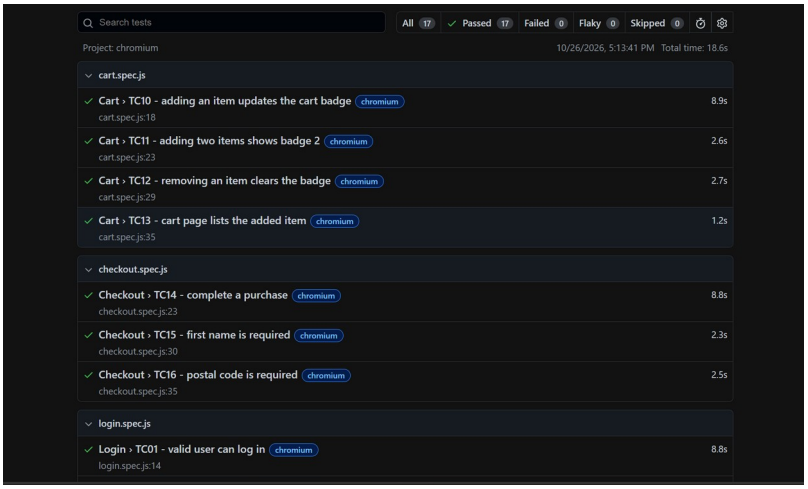


Figure 9: HTML report at localhost:9323 (user manual, step 8)

Step 9: Press Ctrl+C in the terminal to stop showing the report.

A1.5 Reading the results

- A green tick means the test passed. A red cross means it failed.

- For a failed test the report shows the error, a picture, a video and a recording of the steps.

A1.6 Common problems and solutions

Table 15: Troubleshooting

Problem	Solution
“Could not read package.json”	The terminal is in the wrong folder. Go to the folder that contains package.json.
“Executable doesn’t exist”	Run npx playwright install chromium and then npm test again.
Tests fail with network errors	Check that https://www.saucedemo.com opens in a normal browser.
git pull says “Unlink of file failed”	Close any open Word or PDF files from the project and pause OneDrive, then pull again.

APPENDIX-2
SOURCE CODE AND EXPLANATION

This appendix contains all the code of the project, exactly as it is in the repository. For every file there are three parts: why the file is needed, the complete code with line numbers, and a table that explains each line in plain words. The files are shown in the order in which Playwright uses them: first the project files, then the settings, then the data, then the page files, and last the test files. The same code is available at https://github.com/gillmreet01/saucedemo-regression-tests.

Two small ideas help in reading the code. A line that starts with // is a comment, a note for people that the computer ignores. Text inside quote marks, such as 'secret_sauce', is plain text. Everything else is an instruction.

A2.1 How the Files Work Together

Before reading each file, it helps to see how they work together. The two tables below follow one test from the first command to the result. The first follows TC02 (wrong password), a short test. The second follows TC14 (complete purchase), a long test. The file names and line numbers refer to the code shown later in this appendix.

Table 16: What happens when TC02 runs

Step	What happens	File and lines
1	The command npm test starts Playwright, which reads the settings: the tests folder, the website address, the waiting times.	playwright.config.js, lines 5 to 18
2	Playwright finds the Login group of tests and opens a fresh browser page for TC02.	tests/login.spec.js, line 6
3	The “before each test” step makes a LoginPage object and opens the home page of the shop.	tests/login.spec.js, lines 9 to 12; pages/LoginPage.js, lines 11 to 13
4	TC02 starts. It takes the standard user name from the data file.	tests/login.spec.js, line 21; utils/testData.js, line 5
5	The login action types the user name and the wrong password, and clicks the Login button.	pages/LoginPage.js, lines 15 to 19
6	The shop shows a red error message.	The website
7	The test finds the error message box and checks that its text contains “Username and password do not match”. It tries for up to 5 seconds.	tests/login.spec.js, line 22; pages/LoginPage.js, line 8; playwright.config.js, line 7
8	The text is found, so TC02 passes and a green tick is printed. The result is added to the HTML report.	playwright.config.js, lines 9 to 12

Table 17: What happens when TC14 runs

Step	What happens	File and lines
1	The “before each test” step of the Checkout group starts. It makes a LoginPage object and opens the shop.	tests/checkout.spec.js, lines 12 to 14
2	It logs in with the standard user and the password from the data file.	tests/checkout.spec.js, line 15; utils/testData.js, lines 3 to 5
3	It adds the backpack to the cart. The product name is turned into the text used by the shop, “sauce-labs-backpack”.	tests/checkout.spec.js, line 17; pages/InventoryPage.js, lines 16 to 22
4	It opens the cart and clicks Checkout. The checkout form opens.	tests/checkout.spec.js, lines 18 to 20; pages/CartPage.js, lines 9 to 11
5	TC14 fills the form with the customer details and clicks Continue.	tests/checkout.spec.js, line 24; pages/CheckoutPage.js, lines 14 to 19
6	The test checks that the page address contains checkout-step-two, which is the overview page.	tests/checkout.spec.js, line 25
7	It clicks Finish.	tests/checkout.spec.js, line 26; pages/CheckoutPage.js, line 9
8	It checks that the heading says “Thank you for your order!”. If it does, TC14 passes.	tests/checkout.spec.js, line 27; pages/CheckoutPage.js, line 11

A2.2 package.json

Why this file is needed

This file describes the project. It gives the project a name, lists the shortcuts for running tests (npm test and others) and says which tools the project needs. npm reads this file when you type npm install. Without it the shortcuts would not work and the tools would not be installed.

Complete code

```
{
  "name": "saucedemo-regression",
  "version": "1.0.0",
  "description": "Playwright regression tests for SauceDemo",
  "scripts": {
    "test": "playwright test",
    "test:headed": "playwright test --headed",
    "test:login": "playwright test tests/login.spec.js",
    "test:products": "playwright test tests/products.spec.js",
    "test:cart": "playwright test tests/cart.spec.js",
    "test:checkout": "playwright test tests/checkout.spec.js",
    "test:logout": "playwright test tests/logout.spec.js",
    "report": "playwright show-report"
  },
  "keywords": [],
  "author": "Manreet Gill",
  "license": "ISC",
```

```
18     "devDependencies": {
19       "@playwright/test": "^1.63.0"
20     }
21   }
```

Explanation

Table 18: Line-by-line explanation of package.json

Line	Code	What it means
1	`{`	The file is written in JSON, a simple format of names and values. The curly bracket opens the main block.
2	`"name": "saucedemo-regression",`	The name of the project.
3	`"version": "1.0.0",`	The version number. 1.0.0 means the first finished version.
4	`"description": "Playwright regression tests for SauceDemo",`	A short sentence that says what the project is for.
5	`"scripts": {`	“scripts” are shortcuts for long commands. The block starts here.
6	`"test": "playwright test",`	Typing npm test runs playwright test, which runs all the tests.
7	`"test:headed": "playwright test --headed",`	npm run test:headed runs all tests with the browser window visible (--headed).
8	`"test:login": "playwright test tests/login.spec.js",`	npm run test:login runs only the login test file.
9	`"test:products": "playwright test tests/products.spec.js",`	npm run test:products runs only the products test file.
10	`"test:cart": "playwright test tests/cart.spec.js",`	npm run test:cart runs only the cart test file.
11	`"test:checkout": "playwright test tests/checkout.spec.js",`	npm run test:checkout runs only the checkout test file.
12	`"test:logout": "playwright test tests/logout.spec.js",`	npm run test:logout runs only the logout test file.
13	`"report": "playwright show-report"`	npm run report opens the HTML report of the last run.
14	`},`	Closes the block of shortcuts.

Line	Code	What it means
15	`"keywords": [],`	Keywords help people find a package online. The list is empty because this project is not published.
16	`"author": "Manreet Gill",`	The author of the project.
17	`"license": "ISC",`	The licence. ISC is a simple permission to copy and use the code.
18	`"devDependencies": {`	“devDependencies” are tools needed only while developing and testing.
19	`"@playwright/test": "^1.63.0"`	The Playwright test tool. ^1.63.0 means version 1.63.0 or any newer 1.x version.
20	`}`	Closes the list of tools.
21	`}`	Closes the file.

A2.3 .gitignore

Why this file is needed

This small file tells Git which files not to store. Some folders are created again automatically (the downloaded tools and the reports), so there is no need to keep them in the repository. Leaving them out keeps the repository small and clean.

Complete code

```
1   node_modules/
2   test-results/
3   playwright-report/
```

Explanation

Table 19: Line-by-line explanation of .gitignore

Line	Code	What it means
1	`node_modules/`	node_modules is the folder of downloaded packages. It is large and npm install can create it again, so Git does not store it.
2	`test-results/`	test-results holds files made during a run, such as screenshots of failures. It is created again on every run.
3	`playwright-report/`	playwright-report is the HTML report. It is also created again on every run.

A2.4 package-lock.json

Why this file is needed

This file is written by npm, not by hand. It records the exact version of every tool that was installed, together with the address it was downloaded from. When another person runs npm install, they get the same versions, so the project behaves the same on every computer. Because it is a generated file, it is explained in groups of lines.

Complete code

```
{
  "name": "saucedemo-regression",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "saucedemo-regression",
      "version": "1.0.0",
      "license": "ISC",
      "devDependencies": {
        "@playwright/test": "^1.63.0"
      }
    },
    "node_modules/@playwright/test": {
      "version": "1.64.1",
      "resolved":
"https://registry.npmjs.org/@playwright/test/-/test-1.64.1.tgz",
      "integrity":
"sha512-IaYGml/nSOiZWROfPtuIL7neuu9qDOZ3TNvNljUIrhDMxiGTgPjzj/g9p9R
xpS2tSepPDRY+UwAaEeiKtUvIAg==",
      "dev": true,
      "license": "Apache-2.0",
      "dependencies": {
        "playwright": "1.64.1"
      },
      "bin": {
        "playwright": "cli.js"
      },
      "engines": {
        "node": ">=20"
      }
    },
    "node_modules/playwright": {
      "version": "1.64.1",
      "resolved":
"https://registry.npmjs.org/playwright/-/playwright-1.64.1.tgz",
```

```
34        "integrity":
    "sha512-HCrVhHr5QQuwYoswEUGTb7gGp522yflbv/ST8nF4sFWGuLjQCt17Q/EubpG
    GSNKYn1tsTCPZxqHhHCfVURu7UA==",
35        "dev": true,
36        "license": "Apache-2.0",
37        "dependencies": {
38          "playwright-core": "1.64.1"
39        },
40        "bin": {
41          "playwright": "cli.js"
42        },
43        "engines": {
44          "node": ">=20"
45        }
46      },
47      "node_modules/playwright-core": {
48        "version": "1.64.1",
49        "resolved":
    "https://registry.npmjs.org/playwright-core/-/playwright-core-
    1.64.1.tgz",
50        "integrity": "sha512-
    BC4s8y6SEVA6Q1rSxZTto8hI1KUI3xXlHGZgvGqP4IqPdI/a6FX9H29IDNz7vKo9l21
    b6LfYosScPL6M3xxyDQ==",
51        "dev": true,
52        "license": "Apache-2.0",
53        "bin": {
54          "playwright-core": "cli.js"
55        },
56        "engines": {
57          "node": ">=20"
58        }
59      }
60    }
61  }
```

Explanation

Table 20: Line-by-line explanation of package-lock.json

Line	Code	What it means
1-6	`{ "name": "saucedemo-regression", "version": "1.0.0", "lockfileVersion": 3, "requires": true, "packages": {`	The start of the file: the project name and version, the format number of the lock file, and the start of the list of packages.
7-14	`"": { ... "devDependencies": { "@playwright/test": "^1.63.0" } },`	The project itself and the one tool it asks for directly, @playwright/test, version 1.63.0 or newer.

Line	Code	What it means
15-30	`"node_modules/@playwright/test": { "version": "1.64.1", "resolved": ..., "integrity": ..., "dev": true, "license": ..., "dependencies": ..., "bin": ..., "engines": ... },`	The exact version that was installed (1.64.1). “resolved” is the address it was downloaded from. “integrity” is a code that proves the download was not changed. “dev” means it is needed only for development. It needs the package playwright, provides the command playwright, and needs Node.js 20 or newer.
31-46	`"node_modules/playwright": { "version": "1.64.1", ... },`	The package playwright, with the same kind of details. It needs playwright-core.
47-59	`"node_modules/playwright-core": { "version": "1.64.1", ... }`	The package playwright-core, the engine that does the real work. It needs nothing else.
60-61	`} }`	Closes the list of packages and the file.

A2.5 playwright.config.js

Why this file is needed

This is the settings file of Playwright. It says where the tests are, how long to wait, which reports to create, which website address to use, whether to show the browser window, what to keep when a test fails, and which browser to use. Changing one value here changes how all 17 tests run.

Complete code

```
1   // Playwright configuration - controls how all tests run
2   const { defineConfig } = require('@playwright/test');
3
4   module.exports = defineConfig({
5     testDir: './tests',            // where the test files live
6     timeout: 30 * 1000,            // max time for one test
7     expect: { timeout: 5 * 1000 }, // max time for one assertion to
    pass
8     retries: 0,                    // no automatic re-runs (keeps
    results honest)
9     reporter: [
10      ['list'],                                                //
    live output in terminal
11      ['html', { outputFolder: 'playwright-report', open: 'never' }],
    // HTML report
12    ],
13    use: {
14      baseURL: 'https://www.saucedemo.com',
```

```
15      headless: true,
16      screenshot: 'only-on-failure', // attach screenshot when a test
    fails
17      video: 'retain-on-failure',    // keep video only for failed
    tests
18      trace: 'retain-on-failure',    // keep trace only for failed
    tests
19    },
20    projects: [
21      { name: 'chromium', use: { browserName: 'chromium' } },
22    ],
23  });
```

Explanation

Table 21: Line-by-line explanation of playwright.config.js

Line	Code	What it means
1	`// Playwright configuration - controls how all tests run`	A line starting with // is a comment: a note for people. The computer ignores it.
2	`const { defineConfig } = require('@playwright/test');`	Takes the helper called defineConfig from Playwright. require means “load this from another place”.
4	`module.exports = defineConfig({`	Shares the settings so that Playwright can read them. The settings are written between the brackets that follow.
5	`testDir: './tests', // where the test files live`	testDir: the folder where the test files are. Playwright looks only in ./tests.
6	`timeout: 30 * 1000, // max time for one test`	timeout: one test may run for at most 30 seconds (30 × 1000 milliseconds). After that it fails.
7	`expect: { timeout: 5 * 1000 }, // max time for one assertion to pass`	expect timeout: each check inside a test may wait up to 5 seconds for the page to be ready.
8	`retries: 0, // no automatic re-runs (keeps results honest)`	retries: 0 means a failed test is not run again. A failure is shown as a failure.
9	`reporter: [`	reporter: the list of report styles to produce.
10	`['list'], // live output in terminal`	The “list” report prints each test in the terminal as it finishes.

Line	Code	What it means
11	`['html', { outputFolder: 'playwright-report', open: 'never' }], // HTML report`	The “html” report creates a web page in the folder playwright-report. open: never means it does not open by itself.
12	`],`	Closes the list of reports.
13	`use: {`	use: settings that every test uses.
14	`baseURL: 'https://www.saucedemo.com',`	baseURL: the address of the website. Later we can write just / instead of the full address.
15	`headless: true,`	headless: true means the browser runs without showing a window. This is faster.
16	`screenshot: 'only-on-failure', // attach screenshot when a test fails`	screenshot: take a picture of the page, but only when a test fails.
17	`video: 'retain-on-failure', // keep video only for failed tests`	video: keep a video of the test, but only when it fails.
18	`trace: 'retain-on-failure', // keep trace only for failed tests`	trace: keep a recording of every step, but only when it fails. It can be replayed later.
19	`},`	Closes the “use” block.
20	`projects: [`	projects: the list of browsers to test with.
21	`{ name: 'chromium', use: { browserName: 'chromium' } },`	One project named chromium. Chromium is the open-source browser that Google Chrome is built on.
22	`],`	Closes the list of projects.
23	`});`	Closes the settings.

A2.6 utils/testData.js

Why this file is needed

This file keeps the data used by the tests: the user names, the password, the product names and the customer details. Keeping the data in one place means a test never contains a typed password or name. If a value changes, it is changed here only.

Complete code

```
1   // All test data in one place so tests stay clean.
2   module.exports = {
```

```
 3     password: 'secret_sauce',
 4     users: {
 5       standard: 'standard_user',
 6       locked: 'locked_out_user',
 7     },
 8     products: {
 9       backpack: 'Sauce Labs Backpack',
10       bikeLight: 'Sauce Labs Bike Light',
11     },
12     customer: {
13       firstName: 'Manreet',
14       lastName: 'Gill',
15       postalCode: '12345',
16     },
17   };
```

Explanation

Table 22: Line-by-line explanation of utils/testData.js

Line	Code	What it means
1	`// All test data in one place so tests stay clean.`	A comment: all test data is kept here, in one place, so the tests stay short.
2	`module.exports = {`	module.exports shares the data below so other files can use it.
3	`password: 'secret_sauce',`	password: the password used by the demo users of the website.
4	`users: {`	users: the names of the demo users.
5	`standard: 'standard_user',`	standard: a normal user who can log in.
6	`locked: 'locked_out_user',`	locked: a user that the website has blocked.
7	`},`	Closes the users block.
8	`products: {`	products: the names of two products used in the tests.
9	`backpack: 'Sauce Labs Backpack',`	backpack: the full name of the backpack as shown on the website.
10	`bikeLight: 'Sauce Labs Bike Light',`	bikeLight: the full name of the bike light.
11	`},`	Closes the products block.
12	`customer: {`	customer: the details typed into the checkout form.
13	`firstName: 'Manreet',`	firstName of the customer.

Line	Code	What it means
14	`lastName: 'Gill',`	lastName of the customer.
15	`postalCode: '12345',`	postalCode of the customer.
16	`},`	Closes the customer block.
17	`};`	Closes the data.

A2.7 pages/LoginPage.js

Why this file is needed

This is the Page Object for the login page. It records where the boxes and buttons of the login page are, and the two actions a user can do there: open the page and log in. The test files use these actions instead of searching for the buttons themselves.

Complete code

```
1   // Page Object: everything about the login page.
2   class LoginPage {
3     constructor(page) {
4       this.page = page;
5       this.usernameInput = page.locator('[data-test="username"]');
6       this.passwordInput = page.locator('[data-test="password"]');
7       this.loginButton = page.locator('[data-test="login-button"]');
8       this.errorMessage = page.locator('[data-test="error"]');
9     }
10
11    async goto() {
12      await this.page.goto('/');
13    }
14
15    async login(username, password) {
16      await this.usernameInput.fill(username);
17      await this.passwordInput.fill(password);
18      await this.loginButton.click();
19    }
20  }
21
22  module.exports = { LoginPage };
```

Explanation

Table 23: Line-by-line explanation of pages/LoginPage.js

Line	Code	What it means
1	`// Page Object: everything about the login page.`	A comment. This is a Page Object: one class that knows everything about the login page.
2	`class LoginPage {`	A class is a blueprint. LoginPage is the blueprint for the login page.
3	`constructor(page) {`	The constructor runs when a new LoginPage is made. It receives page, which is the browser tab.
4	`this.page = page;`	Remembers the browser tab inside this object.
5	`this.usernameInput = page.locator('[data-test="username"]');`	Finds the user name box on the page using its data-test label and stores it as usernameInput.
6	`this.passwordInput = page.locator('[data-test="password"]');`	Finds the password box and stores it as passwordInput.
7	`this.loginButton = page.locator('[data-test="login-button"]');`	Finds the Login button and stores it as loginButton.
8	`this.errorMessage = page.locator('[data-test="error"]');`	Finds the red error message box and stores it as errorMessage.
9	`}`	Closes the constructor.
11	`async goto() {`	An action called goto. async means it takes time because the browser must load a page.
12	`await this.page.goto('/');`	Opens the home page of the website ("/" with the baseURL). await means wait until the page has loaded.
13	`}`	Closes goto.
15	`async login(username, password) {`	An action called login. It needs a user name and a password.
16	`await this.usernameInput.fill(username);`	Types the user name into the user name box.

Line	Code	What it means
17	`await this.passwordInput.fill(password);`	Types the password into the password box.
18	`await this.loginButton.click();`	Clicks the Login button.
19	`}`	Closes login.
20	`}`	Closes the class.
22	`module.exports = { LoginPage };`	Shares the class so the test files can use LoginPage.

A2.8 pages/InventoryPage.js

Why this file is needed

This is the Page Object for the products page, which is the page shown after login. It also covers the menu and the cart icon. It records the parts of the page and the actions: add to cart, remove from cart, sort, open the cart and log out.

Complete code

```
1   // Page Object: the products page (also has the menu and cart
    icon).
2   class InventoryPage {
3     constructor(page) {
4       this.page = page;
5       this.title = page.locator('[data-test="title"]');
6       this.items = page.locator('[data-test="inventory-item"]');
7       this.itemNames = page.locator('[data-test="inventory-item-
    name"]');
8       this.sortDropdown = page.locator('[data-test="product-sort-
    container"]');
9       this.cartBadge = page.locator('[data-test="shopping-cart-
    badge"]');
10      this.cartLink = page.locator('[data-test="shopping-cart-
    link"]');
11      this.menuButton = page.locator('#react-burger-menu-btn');
12      this.logoutLink = page.locator('[data-test="logout-sidebar-
    link"]');
13    }
14
15    // Turns "Sauce Labs Backpack" into "add-to-cart-sauce-labs-
    backpack"
16    idFor(productName) {
17      return productName.toLowerCase().replace(/ /g, '-');
18    }
```

```
19
20    async addToCart(productName) {
21      await this.page.locator(`[data-test="add-to-cart-$
    {this.idFor(productName)}"]`).click();
22    }
23
24    async removeFromCart(productName) {
25      await this.page.locator(`[data-test="remove-$
    {this.idFor(productName)}"]`).click();
26    }
27
28    async sortBy(value) {
29      await this.sortDropdown.selectOption(value); // az, za, lohi,
    hilo
30    }
31
32    async openCart() {
33      await this.cartLink.click();
34    }
35
36    async logout() {
37      await this.menuButton.click();
38      await this.logoutLink.click();
39    }
40  }
41
42  module.exports = { InventoryPage };
```

Explanation

Table 24: Line-by-line explanation of pages/InventoryPage.js

Line	Code	What it means
1	`// Page Object: the products page (also has the menu and cart icon).`	A comment. This Page Object is for the products page. It also has the menu and the cart icon.
2	`class InventoryPage {`	The blueprint for the products page.
3	`constructor(page) {`	The constructor receives page, the browser tab.
4	`this.page = page;`	Remembers the browser tab.
5	`this.title = page.locator('[data-test="title"]');`	Finds the page title (it shows “Products”).
6	`this.items = page.locator('[data-test="inventory-item"]');`	Finds all the product cards.

Line	Code	What it means
7	`this.itemNames = page.locator('[data-test="inventory-item-name"]');`	Finds the product names.
8	`this.sortDropdown = page.locator('[data-test="product-sort-container"]');`	Finds the box used to choose how products are sorted.
9	`this.cartBadge = page.locator('[data-test="shopping-cart-badge"]');`	Finds the small number on the cart icon.
10	`this.cartLink = page.locator('[data-test="shopping-cart-link"]');`	Finds the cart icon.
11	`this.menuButton = page.locator('#react-burger-menu-btn');`	Finds the menu button (three lines). It is found by its id, which starts with #.
12	`this.logoutLink = page.locator('[data-test="logout-sidebar-link"]');`	Finds the Logout link inside the menu.
13	`}`	Closes the constructor.
15	`// Turns "Sauce Labs Backpack" into "add-to-cart-sauce-labs-backpack"`	A comment explaining the next function.
16	`idFor(productName) {`	A helper called idFor. It turns a product name into the text used by the website for that product.
17	`return productName.toLowerCase().replace(/ /g, '-');`	Changes all letters to small letters and replaces every space with a dash, so “Sauce Labs Backpack” becomes “sauce-labs-backpack”.
18	`}`	Closes idFor.
20	`async addToCart(productName) {`	An action: add a product to the cart.
21	`await this.page.locator(`[data-test="add-to-cart-${this.idFor(productName)}"]`).click();`	Clicks the “add to cart” button of that product. The text inside the backquotes is built from the product id.
22	`}`	Closes addToCart.

Line	Code	What it means
24	`async removeFromCart(productName) {`	An action: remove a product from the cart.
25	`await this.page.locator(`[data-test="remove-${this.idFor(productName)}"]`).click();`	Clicks the “remove” button of that product.
26	`}`	Closes removeFromCart.
28	`async sortBy(value) {`	An action: choose a sort order.
29	`await this.sortDropdown.selectOption(value); // az, za, lohi, hilo`	Chooses an option in the sort box. The values are az, za, lohi and hilo.
30	`}`	Closes sortBy.
32	`async openCart() {`	An action: open the cart.
33	`await this.cartLink.click();`	Clicks the cart icon.
34	`}`	Closes openCart.
36	`async logout() {`	An action: log out.
37	`await this.menuButton.click();`	Clicks the menu button to open the menu.
38	`await this.logoutLink.click();`	Clicks the Logout link.
39	`}`	Closes logout.
40	`}`	Closes the class.
42	`module.exports = { InventoryPage };`	Shares the class so the test files can use InventoryPage.

A2.9 pages/CartPage.js

Why this file is needed

This is the Page Object for the shopping cart page. It is the smallest page file. It records the list of names in the cart and the Checkout button, and has one action: go to checkout.

Complete code

```
1   // Page Object: the shopping cart page.
2   class CartPage {
3     constructor(page) {
4       this.page = page;
```

```
5       this.itemNames = page.locator('[data-test="inventory-item-
    name"]');
6       this.checkoutButton = page.locator('[data-test="checkout"]');
7     }
8
9     async checkout() {
10      await this.checkoutButton.click();
11    }
12  }
13
14  module.exports = { CartPage };
```

Explanation

Table 25: Line-by-line explanation of pages/CartPage.js

Line	Code	What it means
1	`// Page Object: the shopping cart page.`	A comment. This Page Object is for the shopping cart page.
2	`class CartPage {`	The blueprint for the cart page.
3	`constructor(page) {`	The constructor receives page, the browser tab.
4	`this.page = page;`	Remembers the browser tab.
5	`this.itemNames = page.locator('[data-test="inventory-item-name"]');`	Finds the names of the products that are in the cart.
6	`this.checkoutButton = page.locator('[data-test="checkout"]');`	Finds the Checkout button.
7	`}`	Closes the constructor.
9	`async checkout() {`	An action: go to checkout.
10	`await this.checkoutButton.click();`	Clicks the Checkout button.
11	`}`	Closes checkout.
12	`}`	Closes the class.
14	`module.exports = { CartPage };`	Shares the class so the test files can use CartPage.

A2.10 pages/CheckoutPage.js

Why this file is needed

This is the Page Object for the checkout pages: the form where the customer types a name and postal code, the overview page, and the last thank-you page. It has one action that fills the form and clicks Continue.

Complete code

```
1   // Page Object: the three checkout screens (info, overview,
    complete).
2   class CheckoutPage {
3     constructor(page) {
4       this.page = page;
5       this.firstName = page.locator('[data-test="firstName"]');
6       this.lastName = page.locator('[data-test="lastName"]');
7       this.postalCode = page.locator('[data-test="postalCode"]');
8       this.continueButton = page.locator('[data-test="continue"]');
9       this.finishButton = page.locator('[data-test="finish"]');
10      this.errorMessage = page.locator('[data-test="error"]');
11      this.completeHeader = page.locator('[data-test="complete-
    header"]');
12    }
13
14    async fillInfo(first, last, zip) {
15      if (first) await this.firstName.fill(first);
16      if (last) await this.lastName.fill(last);
17      if (zip) await this.postalCode.fill(zip);
18      await this.continueButton.click();
19    }
20  }
21
22  module.exports = { CheckoutPage };
```

Explanation

Table 26: Line-by-line explanation of pages/CheckoutPage.js

Line	Code	What it means
1	`// Page Object: the three checkout screens (info, overview, complete).`	A comment. This Page Object covers the three checkout screens: information, overview and complete.
2	`class CheckoutPage {`	The blueprint for the checkout pages.
3	`constructor(page) {`	The constructor receives page, the browser tab.
4	`this.page = page;`	Remembers the browser tab.

Line	Code	What it means
5	`this.firstName = page.locator('[data-test="firstName"]');`	Finds the first name box.
6	`this.lastName = page.locator('[data-test="lastName"]');`	Finds the last name box.
7	`this.postalCode = page.locator('[data-test="postalCode"]');`	Finds the postal code box.
8	`this.continueButton = page.locator('[data-test="continue"]');`	Finds the Continue button.
9	`this.finishButton = page.locator('[data-test="finish"]');`	Finds the Finish button.
10	`this.errorMessage = page.locator('[data-test="error"]');`	Finds the red error message box.
11	`this.completeHeader = page.locator('[data-test="complete-header"]');`	Finds the heading of the last screen (“Thank you for your order!”).
12	`}`	Closes the constructor.
14	`async fillInfo(first, last, zip) {`	An action: fill the form and click Continue. It needs a first name, a last name and a postal code.
15	`if (first) await this.firstName.fill(first);`	Types the first name, but only if it is not empty. An empty text counts as “nothing”, so the box is skipped.
16	`if (last) await this.lastName.fill(last);`	Types the last name, but only if it is not empty.
17	`if (zip) await this.postalCode.fill(zip);`	Types the postal code, but only if it is not empty.
18	`await this.continueButton.click();`	Clicks the Continue button.
19	`}`	Closes fillInfo.
20	`}`	Closes the class.
22	`module.exports = { CheckoutPage };`	Shares the class so the test files can use CheckoutPage.

A2.11 tests/login.spec.js

Why this file is needed

This file holds the six login tests, TC01 to TC06. They check a correct login, a wrong password, a locked user, an empty user name, an empty password and opening the products page without logging in.

Complete code

```
const { test, expect } = require('@playwright/test');
const { LoginPage } = require('../pages/LoginPage');
const { InventoryPage } = require('../pages/InventoryPage');
const { users, password } = require('../utils/testData');

test.describe('Login', () => {
  let loginPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    await loginPage.goto();
  });

  test('TC01 - valid user can log in', async ({ page }) => {
    await loginPage.login(users.standard, password);
    await expect(page).toHaveURL(/inventory\.html/);
    await expect(new
InventoryPage(page).title).toHaveText('Products');
  });

  test('TC02 - invalid password shows error', async () => {
    await loginPage.login(users.standard, 'wrong_password');
    await expect(loginPage.errorMessage).toContainText('Username
and password do not match');
  });

  test('TC03 - locked out user cannot log in', async () => {
    await loginPage.login(users.locked, password);
    await expect(loginPage.errorMessage).toContainText('this user
has been locked out');
  });

  test('TC04 - empty username shows error', async () => {
    await loginPage.login('', password);
    await expect(loginPage.errorMessage).toContainText('Username is
required');
  });

```

```
35     test('TC05 - empty password shows error', async () => {
36       await loginPage.login(users.standard, '');
37       await expect(loginPage.errorMessage).toContainText('Password is
     required');
38     });
39
40     test('TC06 - cannot open inventory page without logging in',
     async ({ page }) => {
41       await page.goto('/inventory.html');
42       await expect(loginPage.errorMessage).toContainText("You can
     only access '/inventory.html' when you are logged in");
43     });
44   });
```

Explanation

Table 27: Line-by-line explanation of tests/login.spec.js

Line	Code	What it means
1	`const { test, expect } = require('@playwright/test');`	Loads test (to write a test) and expect (to check a result) from Playwright.
2	`const { LoginPage } = require('../pages/LoginPage');`	Loads the LoginPage class from the pages folder.
3	`const { InventoryPage } = require('../pages/InventoryPage');`	Loads the InventoryPage class.
4	`const { users, password } = require('../utils/testData');`	Loads the users and the password from the test data file.
6	`test.describe('Login', () => {`	Starts a group of tests named “Login”. All tests of this file are inside this group.
7	`let loginPage;`	Creates a box named loginPage. The tests of this group share it.
9	`test.beforeEach(async ({ page }) => {`	Runs before every test in this group. page is a fresh browser tab.
10	`loginPage = new LoginPage(page);`	Makes a LoginPage object for that browser tab.
11	`await loginPage.goto();`	Opens the login page of the website.
12	`});`	Closes the beforeEach block.
14	`test('TC01 - valid user can log in', async ({ page }) => {`	Test TC01. The text is its name. The steps are written between the brackets.

Line	Code	What it means
15	`await loginPage.login(users.standard, password);`	Logs in with the standard user and the correct password.
16	`await expect(page).toHaveURL(/inventory\.html/);`	Checks that the page address contains inventory.html. The pattern between the slashes means “contains this text”.
17	`await expect(new InventoryPage(page).title).toHaveText('Products');`	Checks that the page title is exactly “Products”. A new InventoryPage object is made for the products page.
18	`});`	Closes TC01.
20	`test('TC02 - invalid password shows error', async () => {`	Test TC02: wrong password.
21	`await loginPage.login(users.standard, 'wrong_password');`	Logs in with the right user but a wrong password.
22	`await expect(loginPage.errorMessage).toContainText('Username and password do not match');`	Checks that the error message contains the text “Username and password do not match”.
23	`});`	Closes TC02.
25	`test('TC03 - locked out user cannot log in', async () => {`	Test TC03: a locked-out user.
26	`await loginPage.login(users.locked, password);`	Logs in with the locked user and the correct password.
27	`await expect(loginPage.errorMessage).toContainText('this user has been locked out');`	Checks that the error message says the user has been locked out.
28	`});`	Closes TC03.
30	`test('TC04 - empty username shows error', async () => {`	Test TC04: empty user name.
31	`await loginPage.login('', password);`	Logs in with an empty user name (two quote marks with nothing inside).
32	`await expect(loginPage.errorMessage).toContainText('Username is required');`	Checks that the error message contains “Username is required”.

Line	Code	What it means
33	`});`	Closes TC04.
35	`test('TC05 - empty password shows error', async () => {`	Test TC05: empty password.
36	`await loginPage.login(users.standard, '');`	Logs in with the standard user and an empty password.
37	`await expect(loginPage.errorMessage) .toContainText('Password is required');`	Checks that the error message contains “Password is required”.
38	`});`	Closes TC05.
40	`test('TC06 - cannot open inventory page without logging in', async ({ page }) => {`	Test TC06: opening the products page without logging in. It uses page directly.
41	`await page.goto('/inventory.html');`	Goes straight to the products page address without logging in.
42	`await expect(loginPage.errorMessage) .toContainText("You can only access '/inventory.html' when you are logged in");`	Checks that the error message says the page can only be opened after login. Double quotes are used because the text contains single quotes.
43	`});`	Closes TC06.
44	`});`	Closes the group.

A2.12 tests/products.spec.js

Why this file is needed

This file holds the three product tests, TC07 to TC09. They check the number of products and the first product after sorting by name and by price. Before each test, it logs in.

Complete code

```
1   const { test, expect } = require('@playwright/test');
2   const { LoginPage } = require('../pages/LoginPage');
3   const { InventoryPage } = require('../pages/InventoryPage');
4   const { users, password } = require('../utils/testData');
5
6   test.describe('Products', () => {
7     let inventoryPage;
```

```
 8
 9    // Before each test: log in and open the products page
10    test.beforeEach(async ({ page }) => {
11      const loginPage = new LoginPage(page);
12      await loginPage.goto();
13      await loginPage.login(users.standard, password);
14      inventoryPage = new InventoryPage(page);
15    });
16
17    test('TC07 - shows 6 products', async () => {
18      await expect(inventoryPage.items).toHaveCount(6);
19    });
20
21    test('TC08 - sort by name A to Z', async () => {
22      await inventoryPage.sortBy('az');
23      await expect(inventoryPage.itemNames.first()).toHaveText('Sauce
    Labs Backpack');
24    });
25
26    test('TC09 - sort by price low to high', async () => {
27      await inventoryPage.sortBy('lohi');
28      await expect(inventoryPage.itemNames.first()).toHaveText('Sauce
    Labs Onesie');
29    });
30  });
```

Explanation

Table 28: Line-by-line explanation of tests/products.spec.js

Line	Code	What it means
1	`const { test, expect } = require('@playwright/test');`	Loads test and expect from Playwright.
2	`const { LoginPage } = require('../pages/LoginPage');`	Loads the LoginPage class.
3	`const { InventoryPage } = require('../pages/InventoryPage');`	Loads the InventoryPage class.
4	`const { users, password } = require('../utils/testData');`	Loads the users and the password.
6	`test.describe('Products', () => {`	Starts a group of tests named “Products”.
7	`let inventoryPage;`	Creates a box named inventoryPage that the tests of this group share.

Line	Code	What it means
9	`// Before each test: log in and open the products page`	A comment: before each test, log in and open the products page.
10	`test.beforeEach(async ({ page }) => {`	Runs before every test in this group.
11	`const loginPage = new LoginPage(page);`	Makes a LoginPage object.
12	`await loginPage.goto();`	Opens the login page.
13	`await loginPage.login(users.standard, password);`	Logs in with the standard user.
14	`inventoryPage = new InventoryPage(page);`	Makes an InventoryPage object for the products page that is now open.
15	`});`	Closes the beforeEach block.
17	`test('TC07 - shows 6 products', async () => {`	Test TC07: the number of products.
18	`await expect(inventoryPage.items).toHaveCount(6);`	Checks that there are exactly 6 product cards.
19	`});`	Closes TC07.
21	`test('TC08 - sort by name A to Z', async () => {`	Test TC08: sort by name A to Z.
22	`await inventoryPage.sortBy('az');`	Chooses the option az (A to Z) in the sort box.
23	`await expect(inventoryPage.itemNames.first()).toHaveText('Sauce Labs Backpack');`	Checks that the first product name is exactly “Sauce Labs Backpack”. first() picks the first one in the list.
24	`});`	Closes TC08.
26	`test('TC09 - sort by price low to high', async () => {`	Test TC09: sort by price, low to high.
27	`await inventoryPage.sortBy('lohi');`	Chooses the option lohi (low to high).
28	`await expect(inventoryPage.itemNames.first()).toHaveText('Sauce Labs Onesie');`	Checks that the first product is “Sauce Labs Onesie”, the cheapest item.
29	`});`	Closes TC09.

Line	Code	What it means
30	`});`	Closes the group.

A2.13 tests/cart.spec.js

Why this file is needed

This file holds the four cart tests, TC10 to TC13. They check the number on the cart icon after adding and removing items, and the content of the cart page. Before each test, it logs in.

Complete code

```
1   const { test, expect } = require('@playwright/test');
2   const { LoginPage } = require('../pages/LoginPage');
3   const { InventoryPage } = require('../pages/InventoryPage');
4   const { CartPage } = require('../pages/CartPage');
5   const { users, password, products } = require('../utils/testData');
6
7   test.describe('Cart', () => {
8     let inventoryPage;
9
10    // Before each test: log in and open the products page
11    test.beforeEach(async ({ page }) => {
12      const loginPage = new LoginPage(page);
13      await loginPage.goto();
14      await loginPage.login(users.standard, password);
15      inventoryPage = new InventoryPage(page);
16    });
17
18    test('TC10 - adding an item updates the cart badge', async () =>
    {
19      await inventoryPage.addToCart(products.backpack);
20      await expect(inventoryPage.cartBadge).toHaveText('1');
21    });
22
23    test('TC11 - adding two items shows badge 2', async () => {
24      await inventoryPage.addToCart(products.backpack);
25      await inventoryPage.addToCart(products.bikeLight);
26      await expect(inventoryPage.cartBadge).toHaveText('2');
27    });
28
29    test('TC12 - removing an item clears the badge', async () => {
30      await inventoryPage.addToCart(products.backpack);
31      await inventoryPage.removeFromCart(products.backpack);
32      await expect(inventoryPage.cartBadge).toHaveCount(0);
33    });
34
```

```
35    test('TC13 - cart page lists the added item', async ({ page }) =>
    {
36      await inventoryPage.addToCart(products.backpack);
37      await inventoryPage.openCart();
38      const cartPage = new CartPage(page);
39      await expect(page).toHaveURL(/cart\.html/);
40      await
    expect(cartPage.itemNames).toHaveText([products.backpack]);
41    });
42  });
```

Explanation

Table 29: Line-by-line explanation of tests/cart.spec.js

Line	Code	What it means
1	`const { test, expect } = require('@playwright/test');`	Loads test and expect from Playwright.
2	`const { LoginPage } = require('../pages/LoginPage');`	Loads the LoginPage class.
3	`const { InventoryPage } = require('../pages/InventoryPage');`	Loads the InventoryPage class.
4	`const { CartPage } = require('../pages/CartPage');`	Loads the CartPage class.
5	`const { users, password, products } = require('../utils/testData');`	Loads the users, the password and the products from the test data.
7	`test.describe('Cart', () => {`	Starts a group of tests named “Cart”.
8	`let inventoryPage;`	Creates a box named inventoryPage that the tests share.
10	`// Before each test: log in and open the products page`	A comment: before each test, log in and open the products page.
11	`test.beforeEach(async ({ page }) => {`	Runs before every test in this group.
12	`const loginPage = new LoginPage(page);`	Makes a LoginPage object.
13	`await loginPage.goto();`	Opens the login page.
14	`await loginPage.login(users.standard, password);`	Logs in with the standard user.

Line	Code	What it means
15	`inventoryPage = new InventoryPage(page);`	Makes an InventoryPage object for the products page.
16	`});`	Closes the beforeEach block.
18	`test('TC10 - adding an item updates the cart badge', async () => {`	Test TC10: adding one item.
19	`await inventoryPage.addToCart(products.backpack);`	Adds the backpack to the cart.
20	`await expect(inventoryPage.cartBadge).toHaveText('1');`	Checks that the number on the cart icon is 1.
21	`});`	Closes TC10.
23	`test('TC11 - adding two items shows badge 2', async () => {`	Test TC11: adding two items.
24	`await inventoryPage.addToCart(products.backpack);`	Adds the backpack.
25	`await inventoryPage.addToCart(products.bikeLight);`	Adds the bike light.
26	`await expect(inventoryPage.cartBadge).toHaveText('2');`	Checks that the number on the cart icon is 2.
27	`});`	Closes TC11.
29	`test('TC12 - removing an item clears the badge', async () => {`	Test TC12: removing an item.
30	`await inventoryPage.addToCart(products.backpack);`	Adds the backpack.
31	`await inventoryPage.removeFromCart(products.backpack);`	Removes the backpack again.
32	`await expect(inventoryPage.cartBadge).toHaveCount(0);`	Checks that the number of cart badges is 0, which means the badge is not shown.
33	`});`	Closes TC12.

Line	Code	What it means
35	`test('TC13 - cart page lists the added item', async ({ page }) => {`	Test TC13: the cart page. It uses page to check the address.
36	`await inventoryPage.addToCart(products.backpack);`	Adds the backpack.
37	`await inventoryPage.openCart();`	Opens the cart.
38	`const cartPage = new CartPage(page);`	Makes a CartPage object for the cart page that is now open.
39	`await expect(page).toHaveURL(/cart\.html/);`	Checks that the page address contains cart.html.
40	`await expect(cartPage.itemNames).toHaveText([products.backpack]);`	Checks that the list of names in the cart is exactly the backpack. The square brackets mean a list.
41	`});`	Closes TC13.
42	`});`	Closes the group.

A2.14 tests/checkout.spec.js

Why this file is needed

This file holds the three checkout tests, TC14 to TC16. They check a complete purchase and the messages for a missing first name and a missing postal code. Before each test, it logs in, adds the backpack and opens the checkout form.

Complete code

```
 1  const { test, expect } = require('@playwright/test');
 2  const { LoginPage } = require('../pages/LoginPage');
 3  const { InventoryPage } = require('../pages/InventoryPage');
 4  const { CartPage } = require('../pages/CartPage');
 5  const { CheckoutPage } = require('../pages/CheckoutPage');
 6  const { users, password, products, customer } =
    require('../utils/testData');
 7
 8  test.describe('Checkout', () => {
 9    let checkoutPage;
10
11    // Before each test: log in, add a backpack and open the checkout
    form
12    test.beforeEach(async ({ page }) => {
```

```
13      const loginPage = new LoginPage(page);
14      await loginPage.goto();
15      await loginPage.login(users.standard, password);
16      const inventoryPage = new InventoryPage(page);
17      await inventoryPage.addToCart(products.backpack);
18      await inventoryPage.openCart();
19      await new CartPage(page).checkout();
20      checkoutPage = new CheckoutPage(page);
21    });
22
23    test('TC14 - complete a purchase', async ({ page }) => {
24      await checkoutPage.fillInfo(customer.firstName,
    customer.lastName, customer.postalCode);
25      await expect(page).toHaveURL(/checkout-step-two/);
26      await checkoutPage.finishButton.click();
27      await expect(checkoutPage.completeHeader).toHaveText('Thank you
    for your order!');
28    });
29
30    test('TC15 - first name is required', async () => {
31      await checkoutPage.fillInfo('', customer.lastName,
    customer.postalCode);
32      await expect(checkoutPage.errorMessage).toContainText('First
    Name is required');
33    });
34
35    test('TC16 - postal code is required', async () => {
36      await checkoutPage.fillInfo(customer.firstName,
    customer.lastName, '');
37      await expect(checkoutPage.errorMessage).toContainText('Postal
    Code is required');
38    });
39  });
```

Explanation

Table 30: Line-by-line explanation of tests/checkout.spec.js

Line	Code	What it means
1	`const { test, expect } = require('@playwright/test');`	Loads test and expect from Playwright.
2	`const { LoginPage } = require('../pages/LoginPage');`	Loads the LoginPage class.
3	`const { InventoryPage } = require('../pages/InventoryPage');`	Loads the InventoryPage class.

Line	Code	What it means
4	`const { CartPage } = require('../pages/CartPage');`	Loads the CartPage class.
5	`const { CheckoutPage } = require('../pages/CheckoutPage');`	Loads the CheckoutPage class.
6	`const { users, password, products, customer } = require('../utils/testData');`	Loads the users, the password, the products and the customer details.
8	`test.describe('Checkout', () => {`	Starts a group of tests named “Checkout”.
9	`let checkoutPage;`	Creates a box named checkoutPage that the tests share.
11	`// Before each test: log in, add a backpack and open the checkout form`	A comment: before each test, log in, add the backpack and open the checkout form.
12	`test.beforeEach(async ({ page }) => {`	Runs before every test in this group.
13	`const loginPage = new LoginPage(page);`	Makes a LoginPage object.
14	`await loginPage.goto();`	Opens the login page.
15	`await loginPage.login(users.standard, password);`	Logs in with the standard user.
16	`const inventoryPage = new InventoryPage(page);`	Makes an InventoryPage object for the products page.
17	`await inventoryPage.addToCart(products.backpack);`	Adds the backpack to the cart.
18	`await inventoryPage.openCart();`	Opens the cart.
19	`await new CartPage(page).checkout();`	Makes a CartPage object and clicks Checkout.
20	`checkoutPage = new CheckoutPage(page);`	Makes a CheckoutPage object. The checkout form is now open.
21	`});`	Closes the beforeEach block.
23	`test('TC14 - complete a purchase', async ({ page }) => {`	Test TC14: a complete purchase.

Line	Code	What it means
24	`await checkoutPage.fillInfo(customer.firstName, customer.lastName, customer.postalCode);`	Fills the form with the customer details and clicks Continue.
25	`await expect(page).toHaveURL(/checkout-step-two/);`	Checks that the address contains checkout-step-two, which is the overview page.
26	`await checkoutPage.finishButton.click();`	Clicks the Finish button.
27	`await expect(checkoutPage.completeHeader).toHaveText('Thank you for your order!');`	Checks that the heading says “Thank you for your order!”.
28	`});`	Closes TC14.
30	`test('TC15 - first name is required', async () => {`	Test TC15: first name is required.
31	`await checkoutPage.fillInfo('', customer.lastName, customer.postalCode);`	Fills the form but leaves the first name empty.
32	`await expect(checkoutPage.errorMessage).toContainText('First Name is required');`	Checks that the error message contains “First Name is required”.
33	`});`	Closes TC15.
35	`test('TC16 - postal code is required', async () => {`	Test TC16: postal code is required.
36	`await checkoutPage.fillInfo(customer.firstName, customer.lastName, '');`	Fills the form but leaves the postal code empty.
37	`await expect(checkoutPage.errorMessage).toContainText('Postal Code is required');`	Checks that the error message contains “Postal Code is required”.
38	`});`	Closes TC16.
39	`});`	Closes the group.

A2.15 tests/logout.spec.js

Why this file is needed

This file holds the one logout test, TC17. It logs in, logs out and checks that the login page is shown again. Because it has only one test, it does the login inside the test.

Complete code

```
1   const { test, expect } = require('@playwright/test');
2   const { LoginPage } = require('../pages/LoginPage');
3   const { InventoryPage } = require('../pages/InventoryPage');
4   const { users, password } = require('../utils/testData');
5
6   test.describe('Logout', () => {
7     test('TC17 - user can log out and returns to login page', async
    ({ page }) => {
8       const loginPage = new LoginPage(page);
9       await loginPage.goto();
10      await loginPage.login(users.standard, password);
11      await new InventoryPage(page).logout();
12      await expect(loginPage.loginButton).toBeVisible();
13    });
14  });
```

Explanation

Table 31: Line-by-line explanation of tests/logout.spec.js

Line	Code	What it means
1	`const { test, expect } = require('@playwright/test');`	Loads test and expect from Playwright.
2	`const { LoginPage } = require('../pages/LoginPage');`	Loads the LoginPage class.
3	`const { InventoryPage } = require('../pages/InventoryPage');`	Loads the InventoryPage class.
4	`const { users, password } = require('../utils/testData');`	Loads the users and the password.
6	`test.describe('Logout', () => {`	Starts a group of tests named “Logout”. It has only one test, so it needs no beforeEach.
7	`test('TC17 - user can log out and returns to login page', async ({ page }) => {`	Test TC17: logging out.

Line	Code	What it means
8	`const loginPage = new LoginPage(page);`	Makes a LoginPage object.
9	`await loginPage.goto();`	Opens the login page.
10	`await loginPage.login(users.standard, password);`	Logs in with the standard user.
11	`await new InventoryPage(page).logout();`	Opens the menu and clicks Logout.
12	`await expect(loginPage.loginButton).toBeVisible();`	Checks that the Login button is visible, which means the login page is shown again.
13	`});`	Closes TC17.
14	`});`	Closes the group.

A2.16 README.md

Why this file is needed

The README is the first file a visitor sees on GitHub. It says what the project is, how to install and run it, how the folders are arranged and which documents are included. It is not code, so only the complete text is shown.

Complete text

````
1   # SauceDemo Automated Regression Tests (Playwright + JavaScript)
2
3   Automated regression suite for https://www.saucedemo.com covering
4   **login, product listing/sorting, cart, checkout and logout** (17
    test cases).
5
6   ## Setup
7   ```bash
8   npm install
9   npx playwright install chromium   # one-time browser download
10  ```
11
12  ## Run
13  ```bash
14  npm test               # all tests (headless)
15  npm run test:headed    # watch the browser
16  npm run test:login     # or :products :cart :checkout :logout
17  npm run report         # open the HTML report
18  ```
````

```
19  Reports are written to `playwright-report/` (HTML). Failed tests
    also keep a
20  screenshot, video and trace in `test-results/`.
21
22  ## Structure
23  ```
24  playwright.config.js   settings: base URL, reporters, screenshots
25  pages/                 Page Object Model - one class per page
    (locators + actions)
26  tests/                 test files, one per feature (each test logs
    in first)
27  utils/testData.js      users, products, customer details
28  docs/                  test plan and test cases, and the project
    report (Word and PDF)
29  ```
30
31  ## Test cases
32  | ID | Feature | What is checked |
33  |----|---------|-----------------|
34  | TC01-06 | Login | valid login, wrong password, locked user, empty
    username/password, direct URL access blocked |
35  | TC07-09 | Products | 6 items shown, sort A-Z, sort price low-high
    |
36  | TC10-13 | Cart | badge count, add/remove, cart page contents |
37  | TC14-16 | Checkout | full purchase, first name and postal code
    required |
38  | TC17 | Logout | returns to login page |
39
40  ## Documents
41  - [Test plan and test
    cases](docs/SauceDemo_Test_Plan_and_Test_Cases.pdf): scope,
    approach, 17 test cases with steps, expected and actual results,
    and defects/observations.
42  - [Project report](docs/SauceDemo_Project_Report.pdf): full project
    report (introduction, literature survey, design, results,
    conclusion, user manual).
```

A2.17 Quick Reference Tables

These two tables collect the main facts of the code in one place, for quick looking up.

Table 32: All actions in the page files

Page file	Action	What it does
LoginPage	goto()	Opens the home page of the shop
LoginPage	login(username, password)	Types the user name and the password and clicks Login
InventoryPage	idFor(productName)	Turns a product name into the text the shop uses for its buttons
InventoryPage	addToCart(productName)	Clicks the “add to cart” button of a product

Page file	Action	What it does
InventoryPage	removeFromCart(productName)	Clicks the “remove” button of a product
InventoryPage	sortBy(value)	Chooses a sort order (az, za, lohi or hilo)
InventoryPage	openCart()	Clicks the cart icon
InventoryPage	logout()	Opens the menu and clicks Logout
CartPage	checkout()	Clicks the Checkout button
CheckoutPage	fillInfo(first, last, zip)	Fills the form (skipping empty values) and clicks Continue

Table 33: Which test uses which files

Test	Test file	Page files used	Data used
TC01	login.spec.js	LoginPage, InventoryPage	users.standard, password
TC02	login.spec.js	LoginPage	users.standard
TC03	login.spec.js	LoginPage	users.locked, password
TC04	login.spec.js	LoginPage	password
TC05	login.spec.js	LoginPage	users.standard
TC06	login.spec.js	LoginPage	none
TC07	products.spec.js	LoginPage, InventoryPage	users.standard, password
TC08	products.spec.js	LoginPage, InventoryPage	users.standard, password
TC09	products.spec.js	LoginPage, InventoryPage	users.standard, password
TC10	cart.spec.js	LoginPage, InventoryPage	products.backpack
TC11	cart.spec.js	LoginPage, InventoryPage	products.backpack, products.bikeLight
TC12	cart.spec.js	LoginPage, InventoryPage	products.backpack
TC13	cart.spec.js	LoginPage, InventoryPage, CartPage	products.backpack
TC14	checkout.spec.js	LoginPage, InventoryPage, CartPage, CheckoutPage	products.backpack, customer
TC15	checkout.spec.js	LoginPage, InventoryPage, CartPage, CheckoutPage	customer
TC16	checkout.spec.js	LoginPage, InventoryPage, CartPage, CheckoutPage	customer
TC17	logout.spec.js	LoginPage, InventoryPage	users.standard, password

APPENDIX-3
ACHIEVEMENTS

- Built a complete set of 17 automatic test cases for SauceDemo using Playwright and JavaScript.
- Got 17 tests passed and 0 failed in the final run, which took 18.6 seconds.
- Used one file for each page, one file for the data and simple checks that wait by themselves.
- Created terminal and HTML reports, with pictures, videos and recordings for failures.
- Compared tools and designs with simple point tables and chose the best design. Simplified the first 23-test version into one that can be fully explained.
- Wrote a test plan with test cases, this project report with every line of code explained, a demo guide and a viva question sheet.
- Put the project on GitHub with a clear history of changes, a lock file and documents.
- Solved real set-up problems: wrong folder, missing browser download, Git remote errors and a locked file.